Agents IA

Fondations, tool use, patterns, déploiement et Model Context Protocol

bstorm TRAINING

Programme de la théorie

Bloc	Contenu
1A · Agent ou workflow	Rappel LLM, définition, boucle, six briques, cas d'usage
1B · Anatomie d'un agent	Modèle, messages, prompt, outils, contexte, mémoire, RAG
1C · Patterns	Chaînage, routage, orchestrateur, multi-agents
1D · Fiabilité et garde-fous	Permissions, validation humaine, injection, évals, coûts
1E · Du poste local au déploiement	Surfaces, pile technique, hébergement, exploitation
2 · MCP	Principe, primitives, transports, Inspector, sécurité
Synthèse	Repères pour choisir une architecture

bstorm TRAINING

SECTION

1A

Agent ou workflow

Ce qu'est un agent, ce qui le distingue d'un workflow, et quand s'en passer

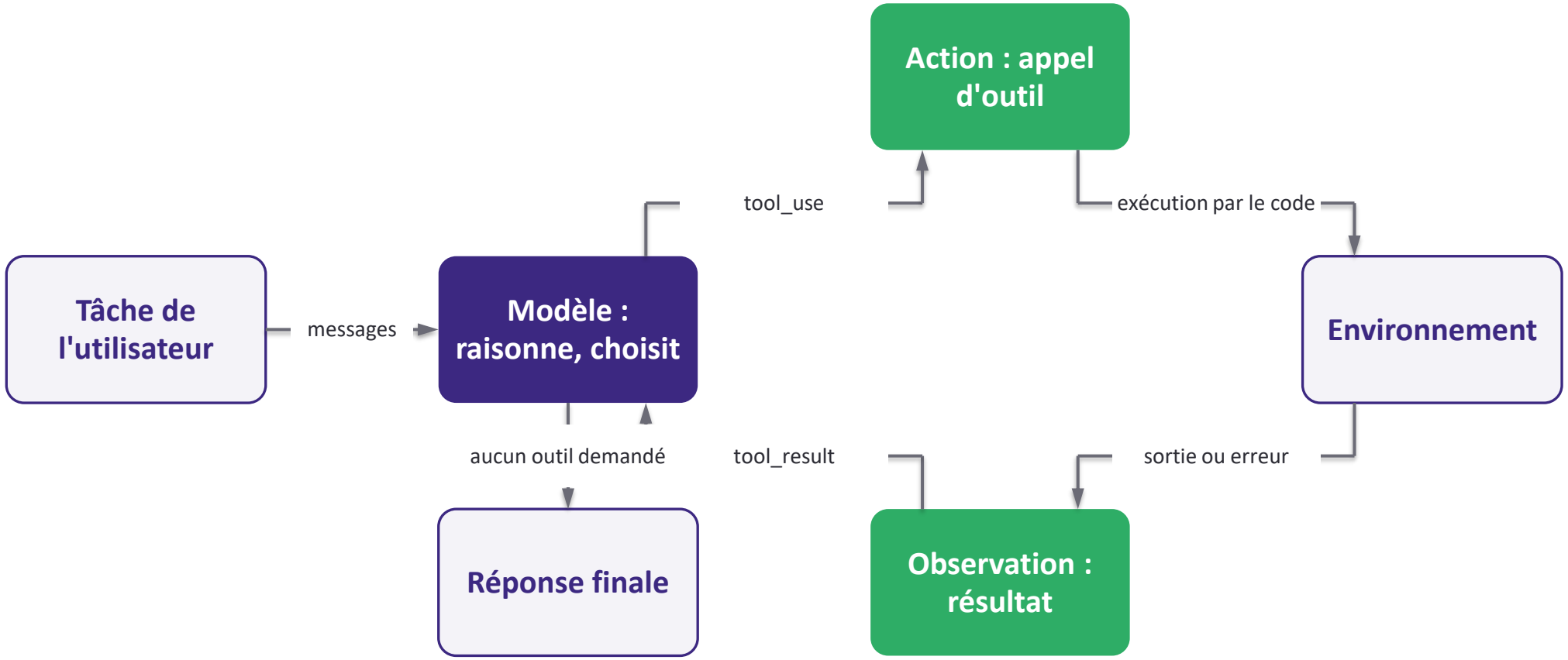
Rappel : ce que fait un modèle de langage

- Entrée et sortie en tokens : fragments de mots, quelques caractères chacun en moyenne
- Génération token par token : chaque token produit dépend de tout ce qui précède dans la fenêtre
- Fenêtre de contexte : nombre maximal de tokens traités en un appel (prompt, historique, outils, réponse)
- Aucune mémoire entre deux appels : l'application renvoie tout l'historique utile à chaque requête
- Connaissances figées à la date de coupure de l'entraînement ; le reste doit être fourni dans le contexte
- Sortie probabiliste : deux appels identiques peuvent donner deux réponses différentes
- Coût et latence proportionnels aux tokens d'entrée et de sortie
- Le modèle ne fait que produire du texte : toute action réelle passe par du code qui l'exécute

Workflow ou agent

Critère	Workflow	Agent
Qui décide du chemin	Le code, écrit d'avance	Le modèle, à chaque étape
Nombre d'étapes	Connu et fixe	Variable, découvert en cours de route
Prévisibilité	Élevée, résultats reproductibles	Plus faible, trajectoires différentes
Coût et latence	Maîtrisés, bornés	Plus élevés, difficiles à prévoir
Tests	Classiques, étape par étape	Évaluation de bout en bout sur cas réels
Tâches adaptées	Bien définies, découpables	Ouvertes, chemin impossible à coder

La boucle d'un agent



Les six briques d'un agent

Objectif
Un résultat visé, pas une procédure

Planification
ReAct : raisonner, agir, observer

Outils
Agir et lire hors du modèle

Agent : modèle dans une boucle
Les six briques réunies par le harnais

Mémoire
Court terme et long terme

Auto-vérification
Tester son résultat avant de conclure

Humain dans la boucle
Valide les actions sensibles

bstorm TRAINING

Avant de construire un agent

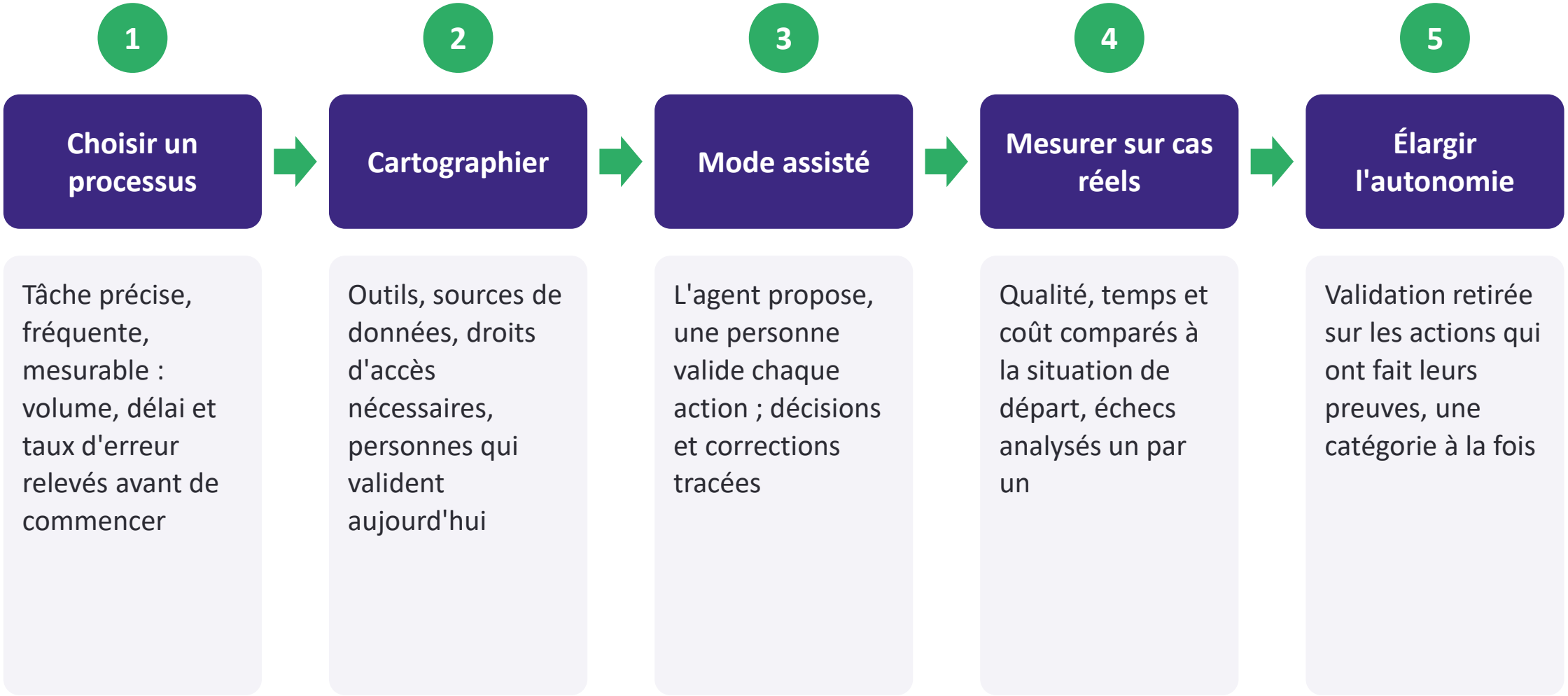
- ☐ La tâche est-elle trop ouverte pour être codée sous forme d'arbre de décision ?
- ☐ Un appel unique bien préparé, avec recherche et exemples, est-il insuffisant ?
- ☐ La valeur d'une tâche réussie justifie-t-elle un coût de plusieurs dizaines d'appels ?
- ☐ Le modèle maîtrise-t-il déjà les capacités clés de la tâche, testées une par une ?
- ☐ Une erreur est-elle détectable et réparable à faible coût ?
- ☐ Le résultat est-il vérifiable : tests, critères objectifs, relecture rapide ?
- ☐ Les outils nécessaires existent-ils et peuvent-ils être limités au strict besoin ?

Cas d'usage en entreprise (1/2)

Autonomie réaliste : ce que l'agent fait seul et ce qui reste validé par une personne.

Cas d'usage	Travail de l'agent	Autonomie réaliste
Support client niveau 1	Répond aux questions courantes depuis la base d'aide	Réponse directe ; transfert à un humain si doute
Tri des courriels	Classe, résume, propose un brouillon de réponse	Classement seul ; envoi validé
Factures fournisseurs	Extrait les champs, rapproche facture et commande	Saisie préparée ; écarts traités par un humain
Analyse de contrats	Repère clauses, échéances, écarts au modèle type	Aide à la lecture ; avis final juridique
Veille marchés publics	Parcourt les avis, filtre selon le profil, résume	Élevée ; décision de répondre humaine
Développement logiciel	Lit le code, modifie, lance les tests, ouvre une PR	Élevée en bac à sable ; fusion après revue

Démarrer un projet d'agent en cinq étapes



bstorm TRAINING

SECTION

1B

Anatomie d'un agent

Modèle, messages, prompt système, outils, contexte et mémoire

Le modèle, cerveau de l'agent

Catégorie	Capacité	Coût et latence	Usage type dans un agent
Grand modèle	Raisonnement long, planification	Élevés	Orchestrateur, tâches difficiles
Modèle intermédiaire	Bon équilibre, code, outils	Moyens	Agent principal par défaut
Petit modèle	Tâches simples et ciblées	Faibles	Routage, classification, exécutants
Modèle ouvert local	Variable selon la taille	Matériel local, pas de facture	Prototypage, données sur le poste
Raisonnement (thinking)	Réflexion avant la réponse	Tokens de sortie en plus	Niveau d'effort réglé selon la tâche

bstorm TRAINING

Messages, rôles et API sans état

Requête

- model, max_tokens : modèle visé et plafond de tokens de sortie
- system : prompt système, paramètre séparé des messages
- messages : liste alternée de rôles user et assistant
- content : texte ou liste de blocs typés (text, image, tool_use, tool_result)
- tools : définitions des outils disponibles
- Tout l'historique utile est renvoyé à chaque appel

Réponse

- role : toujours assistant
- content : liste de blocs, text et éventuellement tool_use
- stop_reason : end_turn, tool_use, max_tokens, stop_sequence…
- usage : tokens d'entrée et de sortie consommés
- La réponse est ajoutée telle quelle à messages avant l'appel suivant

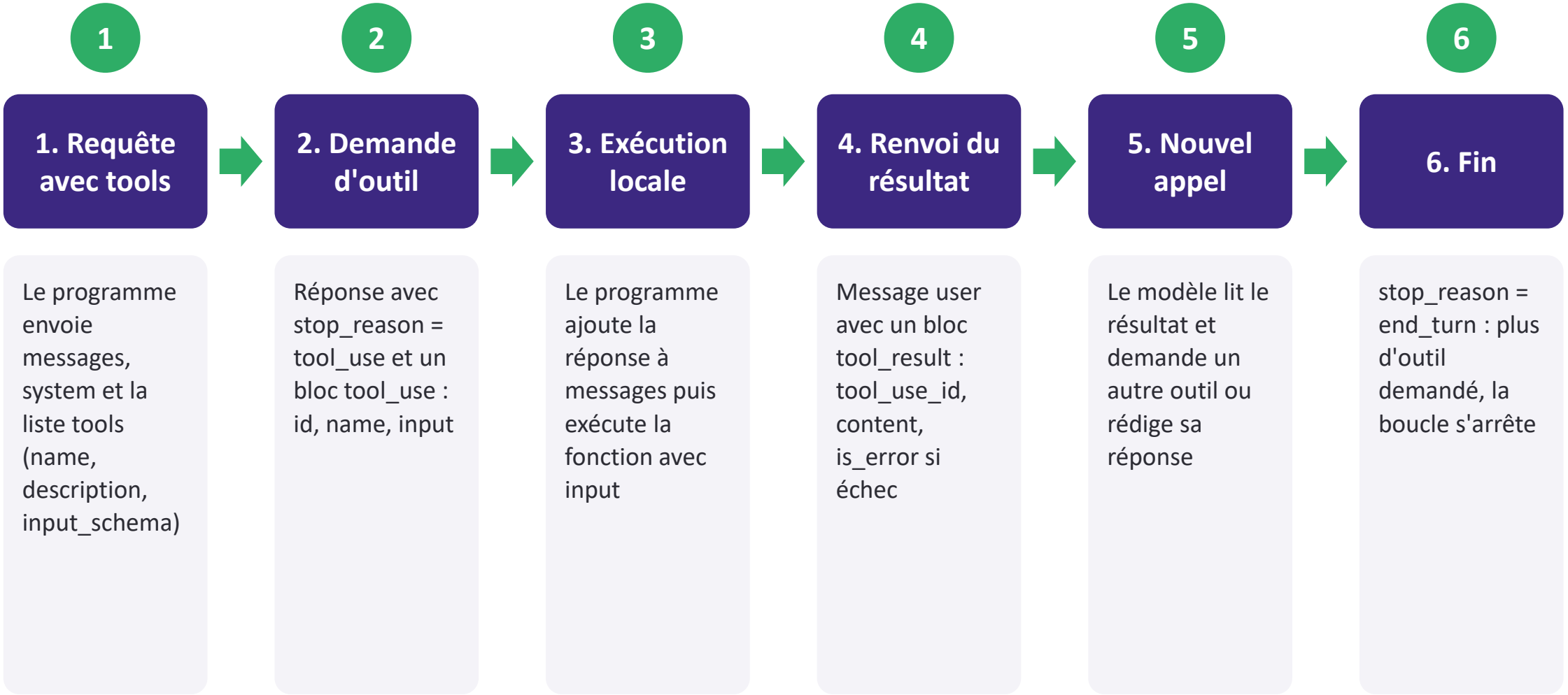
bstorm TRAINING

Le prompt système d'un agent

- Rôle et objectif : ce que l'agent doit accomplir et pour quel résultat
- Contexte stable : domaine, conventions, contraintes de l'environnement
- Outils : quand utiliser chacun, dans quel ordre, quoi faire en cas d'erreur
- Critères de fin : à quel moment la tâche est considérée comme terminée
- Limites : actions interdites ou soumises à validation, budget d'appels
- Format de sortie attendu pour la réponse finale
- Bon niveau de précision : des principes clairs plutôt qu'une liste de cas fragile
- Sections nettes, balises XML ou titres Markdown, et quelques exemples typiques
- Version minimale d'abord, puis ajouts guidés par les échecs observés

bstorm TRAINING

Cycle tool_use / tool_result



bstorm TRAINING

Boucle minimale avec un outil

```python
import anthropic
client = anthropic.Anthropic()          # clé lue dans ANTHROPIC_API_KEY
MODEL = "claude-sonnet-5-5"             # à remplacer par le modèle disponible
TOOLS = [{"name": "get_weather", "description": "Météo actuelle d'une ville.",
          "input_schema": {"type": "object", "required": ["city"],
                           "properties": {"city": {"type": "string"}}}}]
def run_tool(name, args):               # exécution locale, ici simulée
    return f"18 °C, ciel couvert à {args['city']}" if name == "get_weather" else "inconnu"
msgs = [{"role": "user", "content": "Faut-il un parapluie à Lille ?"}]
for _ in range(10):                     # limite de tours
    r = client.messages.create(model=MODEL, max_tokens=1024, tools=TOOLS, messages=msgs)
    msgs.append({"role": "assistant", "content": r.content})
    if r.stop_reason != "tool_use":
        break
    msgs.append({"role": "user", "content": [
        {"type": "tool_result", "tool_use_id": b.id, "content": run_tool(b.name, b.input)}
        for b in r.content if b.type == "tool_use"]})
print("".join(b.text for b in r.content if b.type == "text"))
```

La réponse du modèle est ajoutée telle quelle ; tous les résultats d'outils repartent dans un seul message user.

bstorm TRAINING

Schéma d'un outil (JSON Schema)

```python
search_orders = {
    "name": "orders_search",
    "description": (
        "Recherche les commandes d'un client. À utiliser avant toute question sur "
        "un retard ou un remboursement. Renvoie au plus `limit` commandes, "
        "les plus récentes d'abord. Ne modifie aucune donnée."),
    "input_schema": {
        "type": "object",
        "properties": {
            "customer_email": {"type": "string", "description": "E-mail exact du client"},
            "status": {"type": "string", "enum": ["pending", "shipped", "delivered"],
                       "description": "Filtre facultatif sur le statut"},
            "limit": {"type": "integer", "minimum": 1, "maximum": 20, "default": 5},
        },
        "required": ["customer_email"],
    },
}
```

Le modèle ne voit que le nom, la description et le schéma : c'est toute la documentation de l'outil.

bstorm TRAINING

Écrire de bons outils

- ☑ Viser une tâche complète plutôt qu'emballer chaque point d'accès d'une API
- ☑ Regrouper sous des préfixes cohérents : crm_search, crm_update, tickets_search
- ☑ Nommer les paramètres sans ambiguïté : user_id plutôt que user
- ☑ Renvoyer des informations utiles au modèle : noms lisibles plutôt qu'identifiants techniques
- ☑ Limiter la taille des résultats : pagination, filtres, troncature avec valeurs par défaut
- ☑ Proposer un format concis et un format détaillé quand les usages diffèrent
- ☑ Écrire des messages d'erreur qui indiquent comment corriger l'appel
- ☑ Mesurer sur des tâches réalistes et lire les traces pour réviser descriptions et schémas

Ingénierie du contexte

Choix et maintien, à chaque tour, du plus petit ensemble de tokens à fort signal dans la fenêtre : prompt système, outils, exemples, historique, résultats d'outils, données externes. Le contexte est une ressource finie au rendement décroissant.

Exemple Un agent qui a lu 40 fichiers garde leurs chemins et un résumé de ce qu'il en a appris, pas leur contenu intégral.

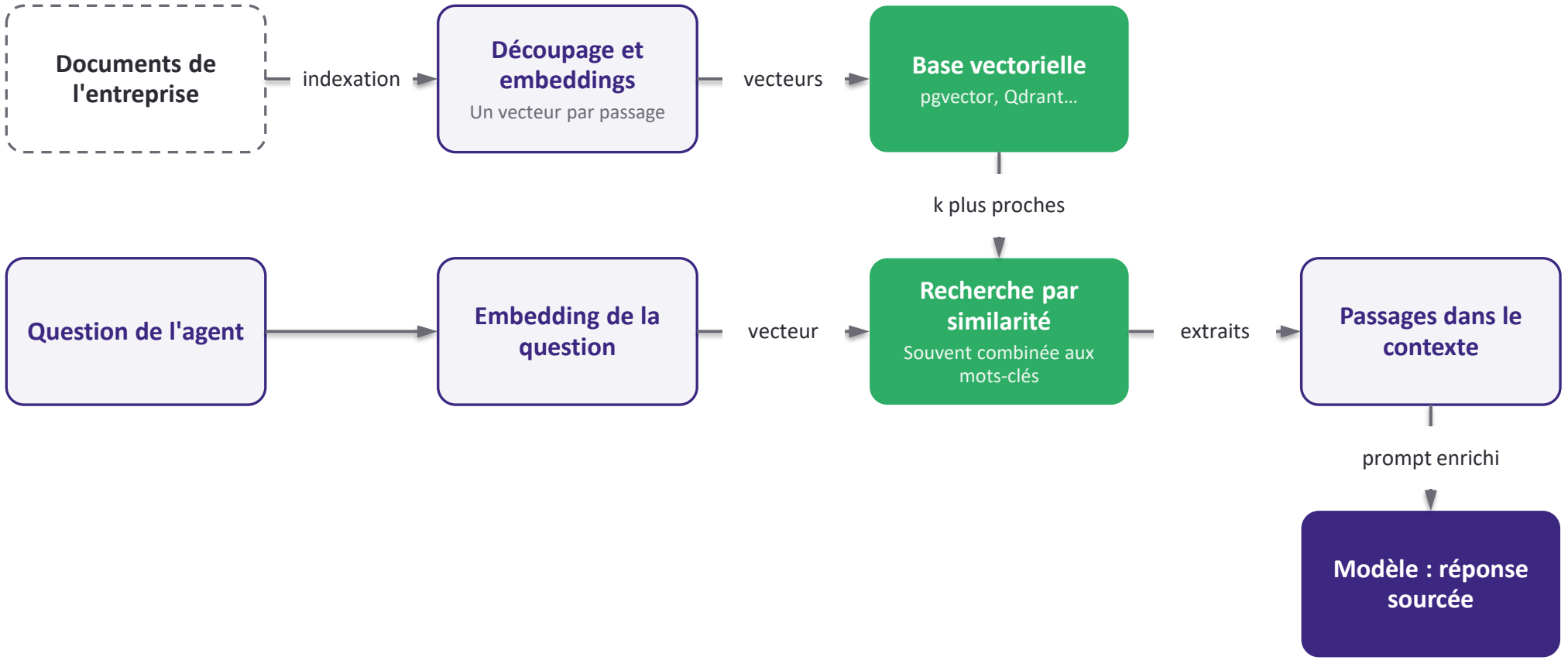
bstorm TRAINING

Mémoire : quatre niveaux

Niveau	Support	Durée de vie	Exemple
Court terme	Fenêtre de contexte, liste messages	Une exécution	Résultats d'outils du tour en cours
Fichiers	Notes lues et écrites par l'agent	Entre exécutions	Fichier de progression, liste de tâches
Long terme	Base externe interrogée par outil	Durable	Préférences, historique client, faits validés
Skills	Dossier avec SKILL.md, scripts, modèles	Durable, versionnée	Procédure métier chargée si pertinente

bstorm TRAINING

Connaissance : RAG et base vectorielle



bstorm TRAINING

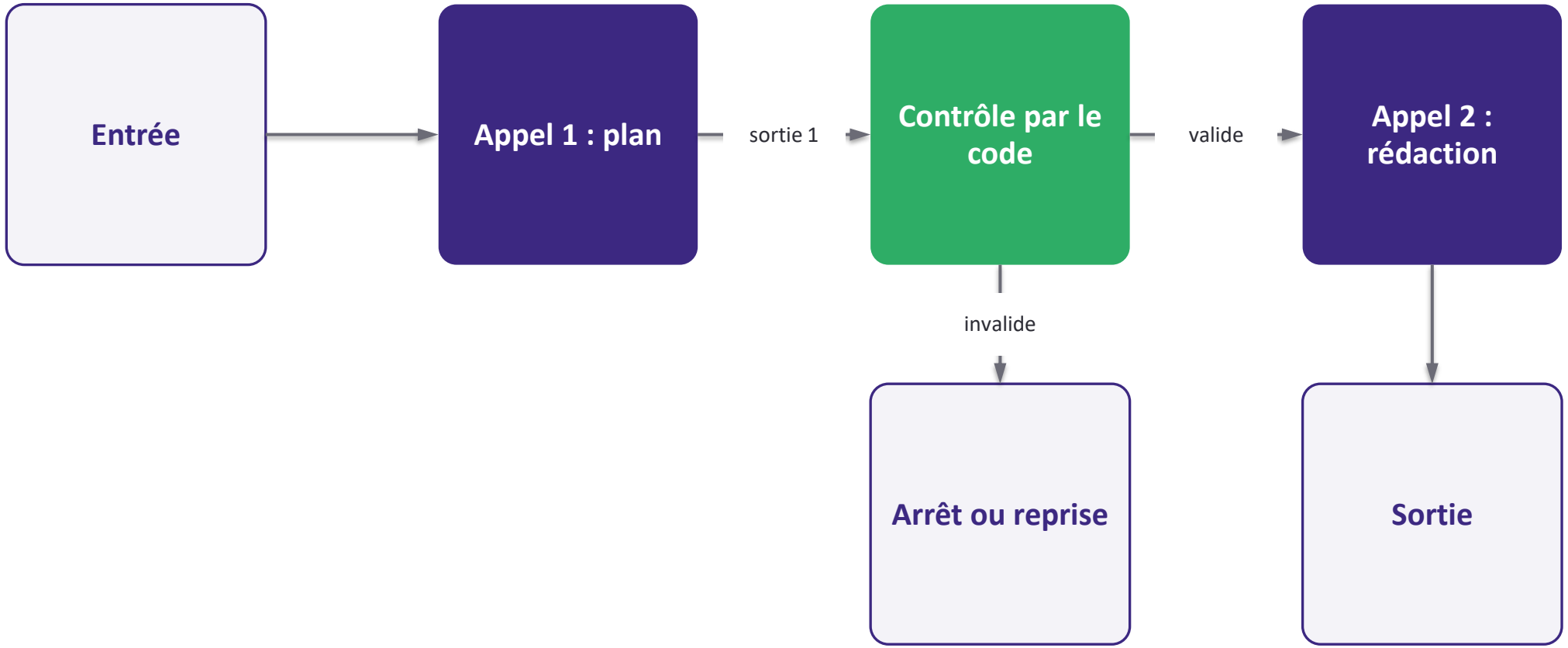
SECTION

1C

Patterns

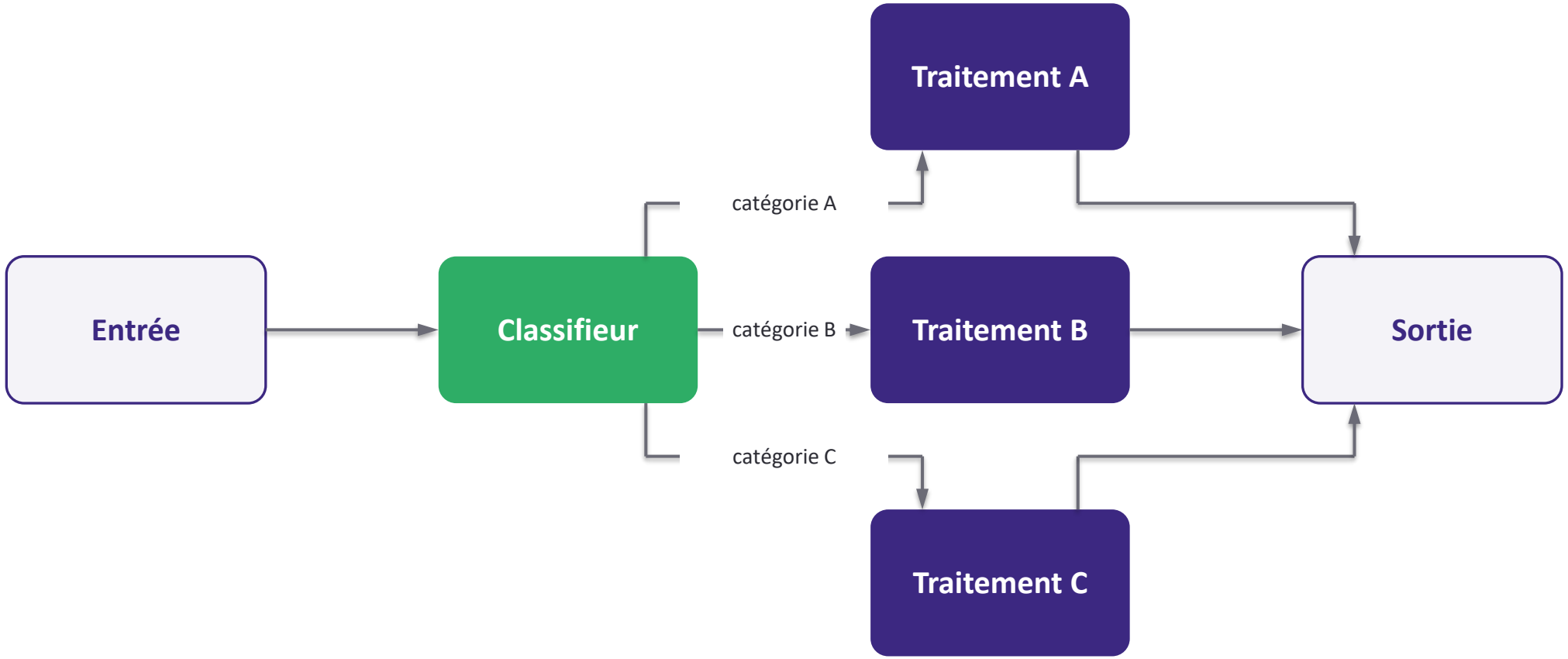
Cinq workflows composables, l'agent autonome et les architectures multi-agents

Chaînage de prompts



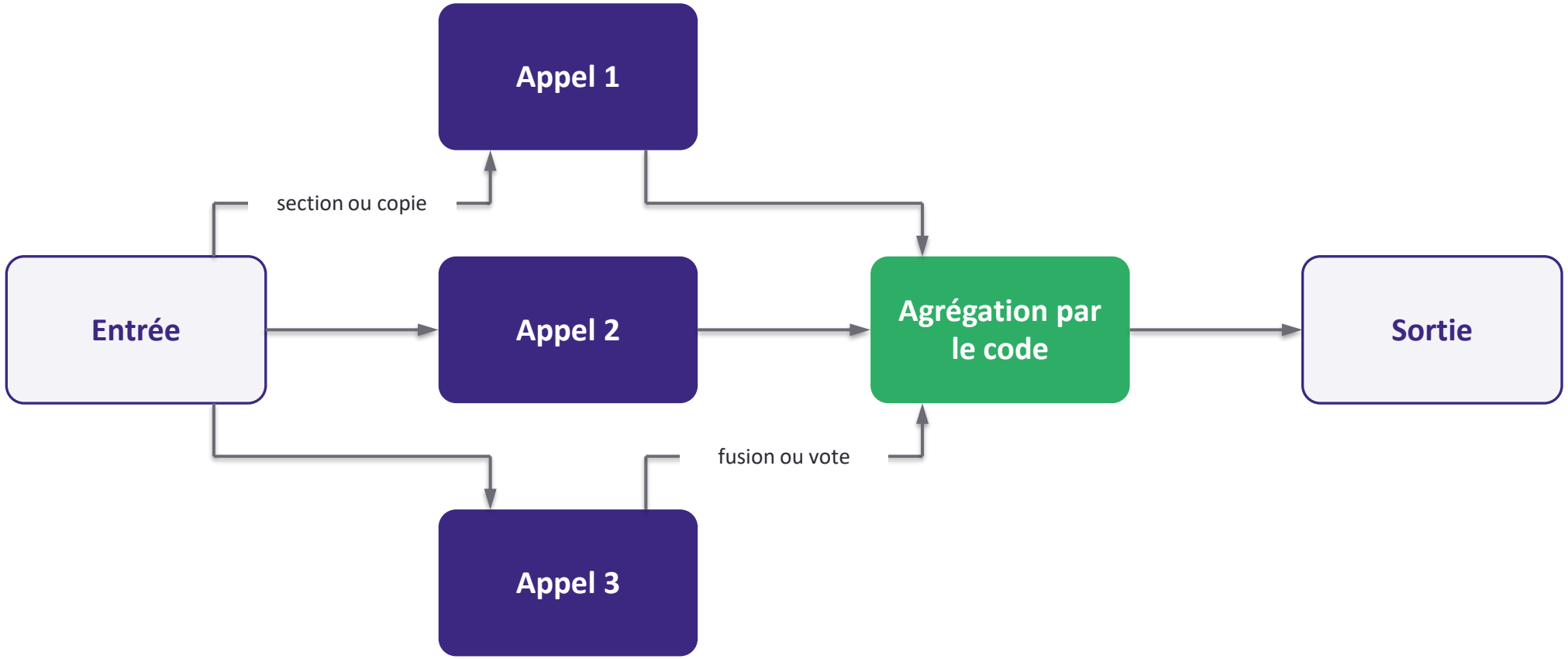
bstorm TRAINING

Routage



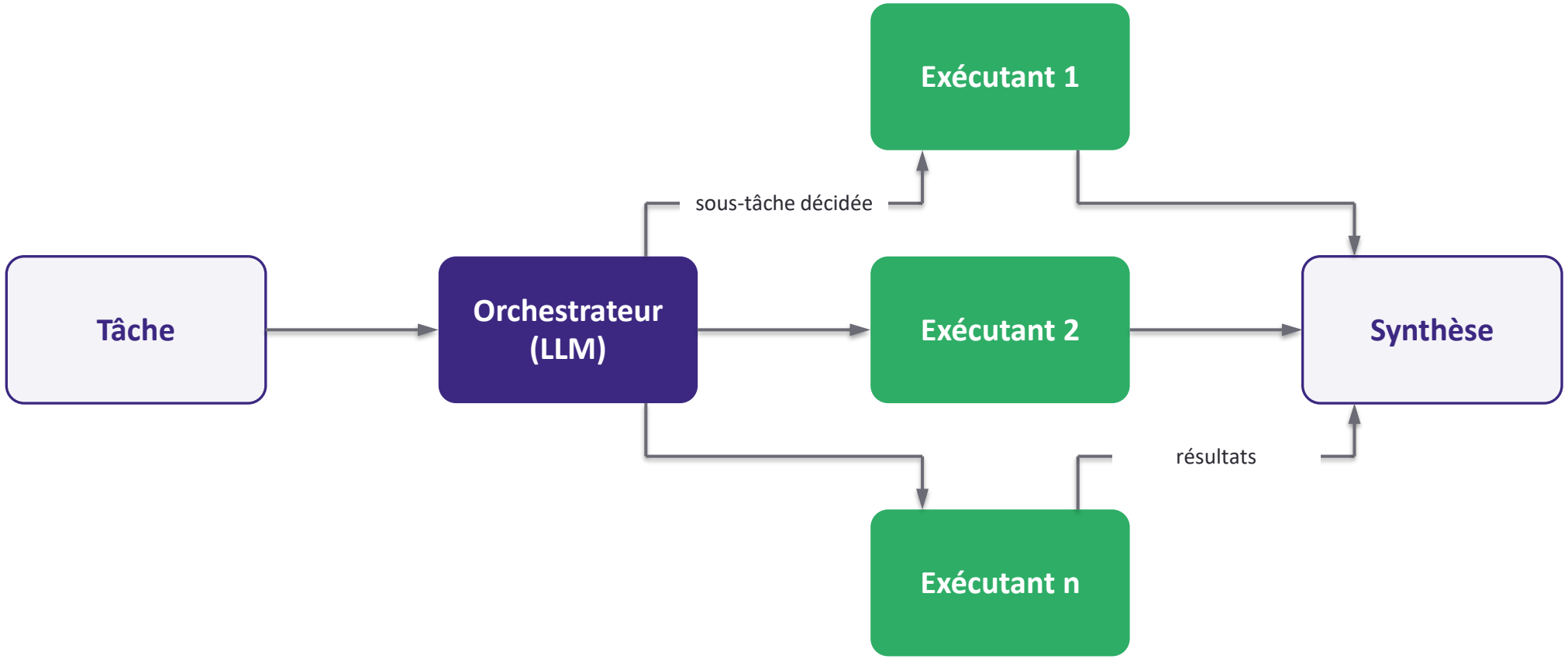
bstorm TRAINING

Parallélisation : sectionnement et vote



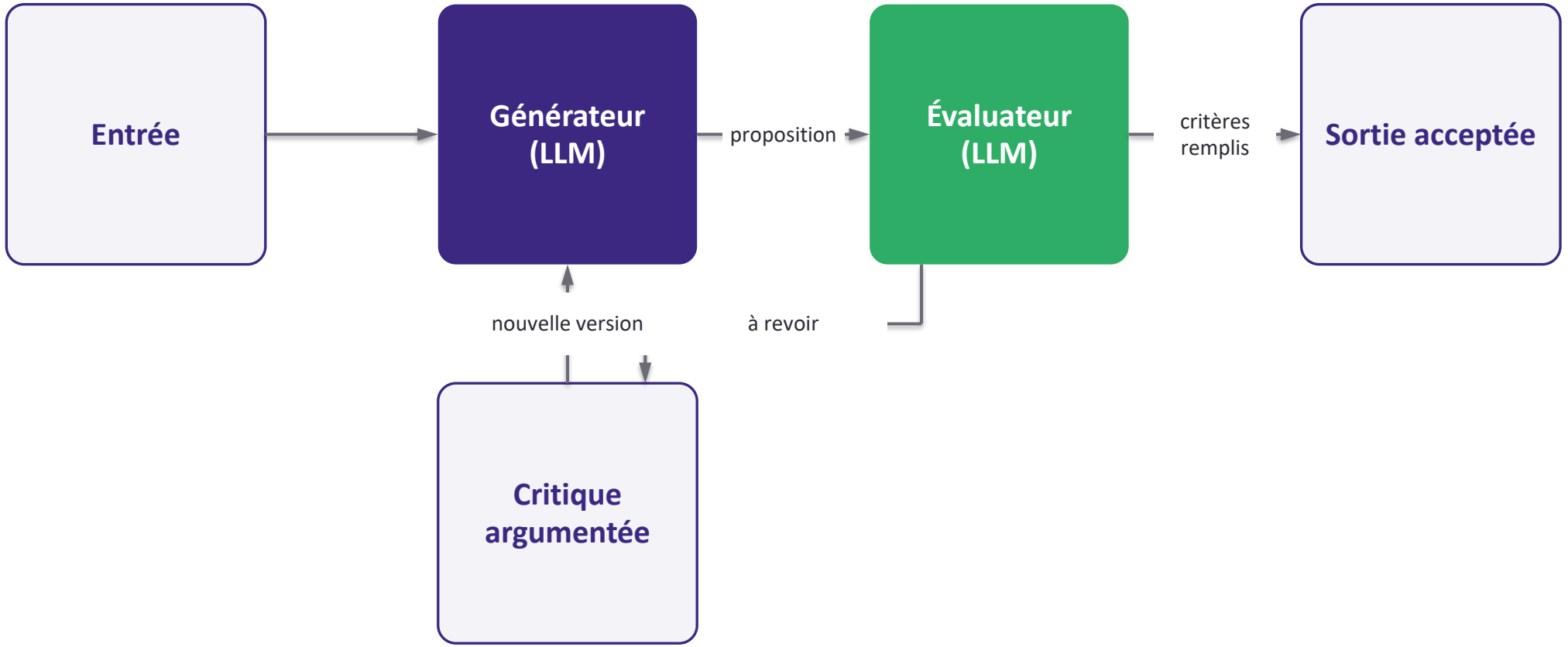
bstorm TRAINING

Orchestrateur et exécutants



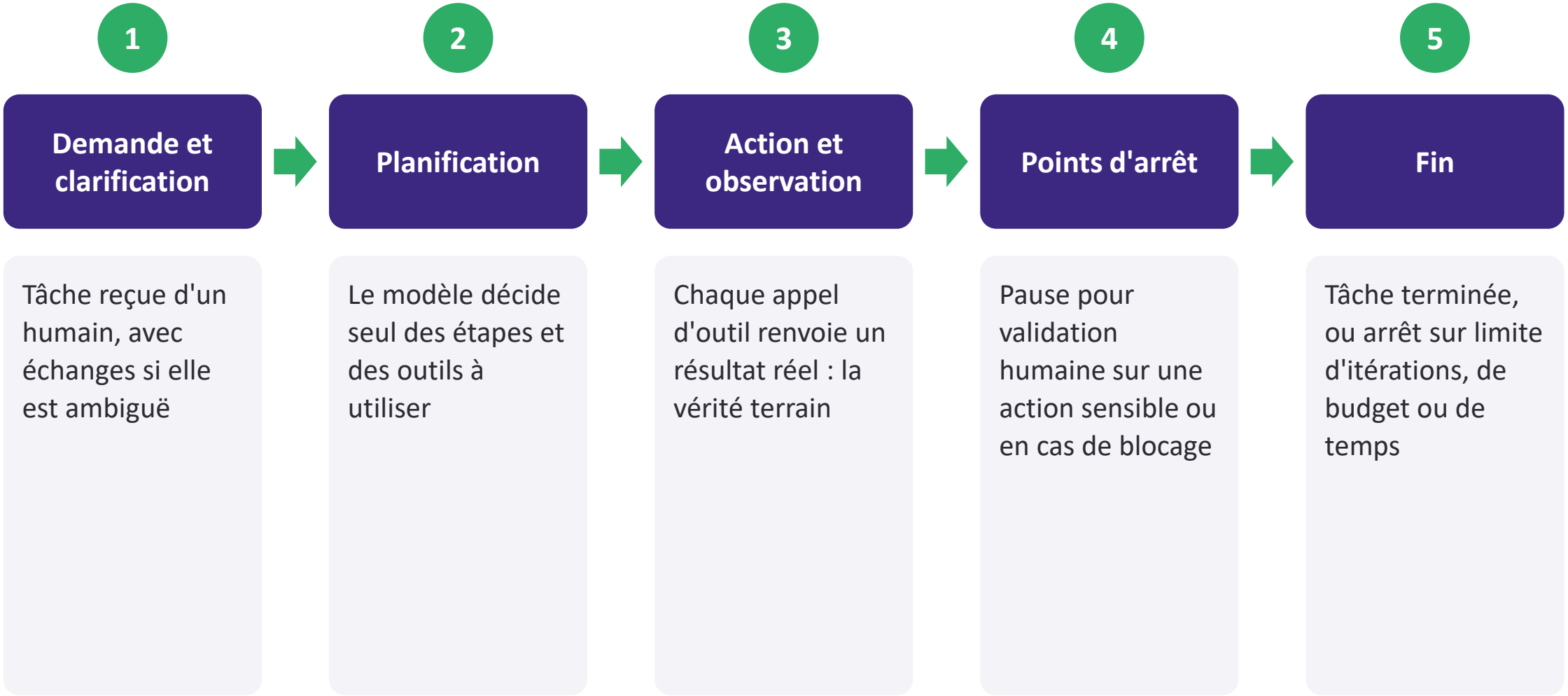
bstorm TRAINING

Évaluateur et optimiseur



bstorm TRAINING

Agent autonome



bstorm TRAINING

Multi-agents : intérêt, coût, cas à éviter

Intérêt

- Contextes séparés : chaque sous-agent explore sans encombrer l'agent principal
- Exploration en parallèle de pistes indépendantes
- Spécialisation : prompt, outils et modèle propres à chaque rôle
- Plus de tokens mobilisés au total, donc plus de capacité de recherche
- Gain net sur les recherches larges qui se découpent bien

Coût et cas à éviter

- Consommation de tokens nettement supérieure à un agent unique
- Coordination délicate : doublons, consignes vagues, sous-agents en trop
- À éviter si tous les agents doivent partager le même contexte
- À éviter si les sous-tâches dépendent fortement les unes des autres
- La plupart des tâches de code se parallélisent mal
- Débogage et évaluation plus difficiles, comportements émergents

Outil, skill ou sous-agent

Critère	Outil	Skill	Sous-agent
Nature	Fonction exécutée par le code	Instructions et scripts en fichiers	Agent avec son propre contexte
Apporte	Une action ou une donnée	Un savoir-faire, une procédure	Un travail délégué complet
Coût en contexte	Schéma toujours chargé	Description seule, corps à la demande	Seul le résumé final revient
Choisir si	Opération précise, déterministe	Méthode répétable, propre au métier	Exploration longue, isolable
Limite	Trop d'outils brouillent le choix	Exige rédaction et mise à jour	Coût en tokens, perte de détails
Exemple	orders_search	Rédiger un rapport d'incident	Analyser un dépôt de code entier

bstorm TRAINING

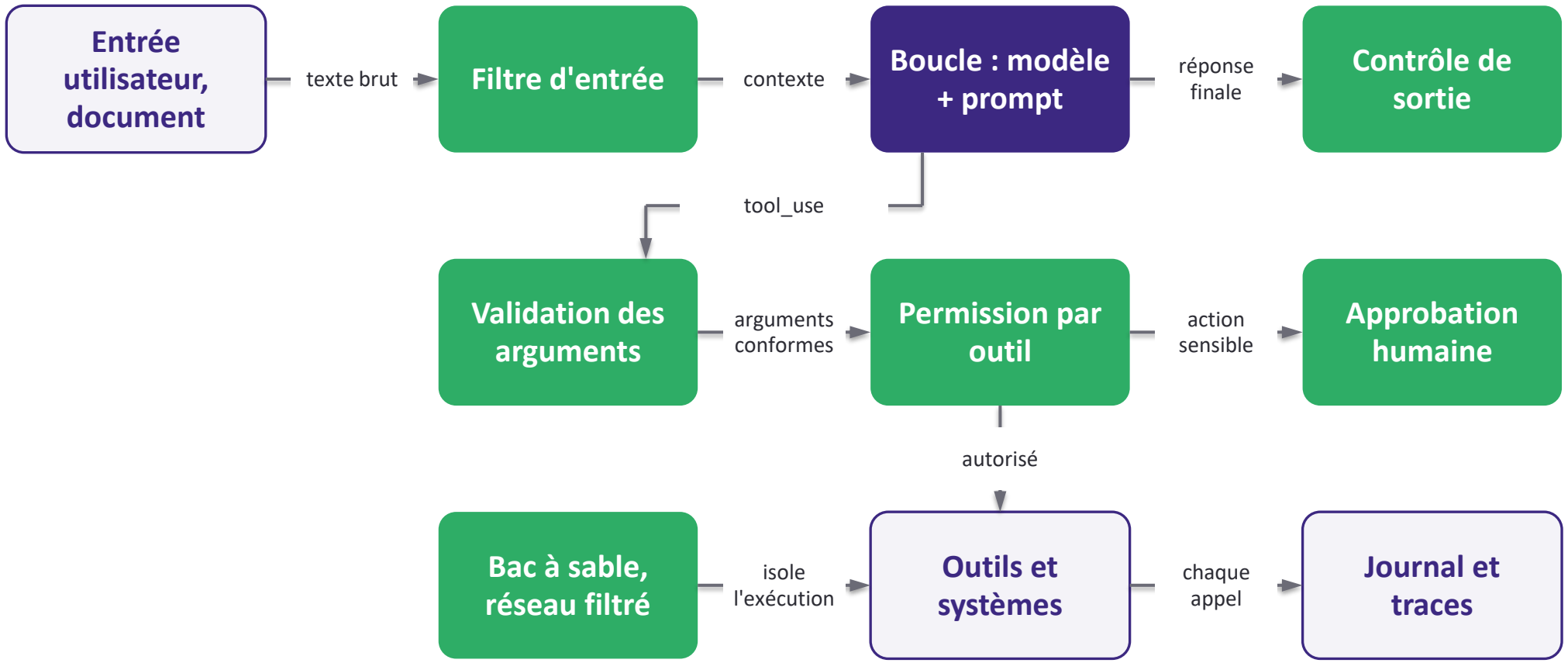
SECTION

1D

Fiabilité et garde-fous

Couches de protection, permissions, validation humaine, injection de prompt, évaluations, traces, coûts

Garde-fous en couches



bstorm TRAINING

Permissions et moindre privilège

Restreindre les outils

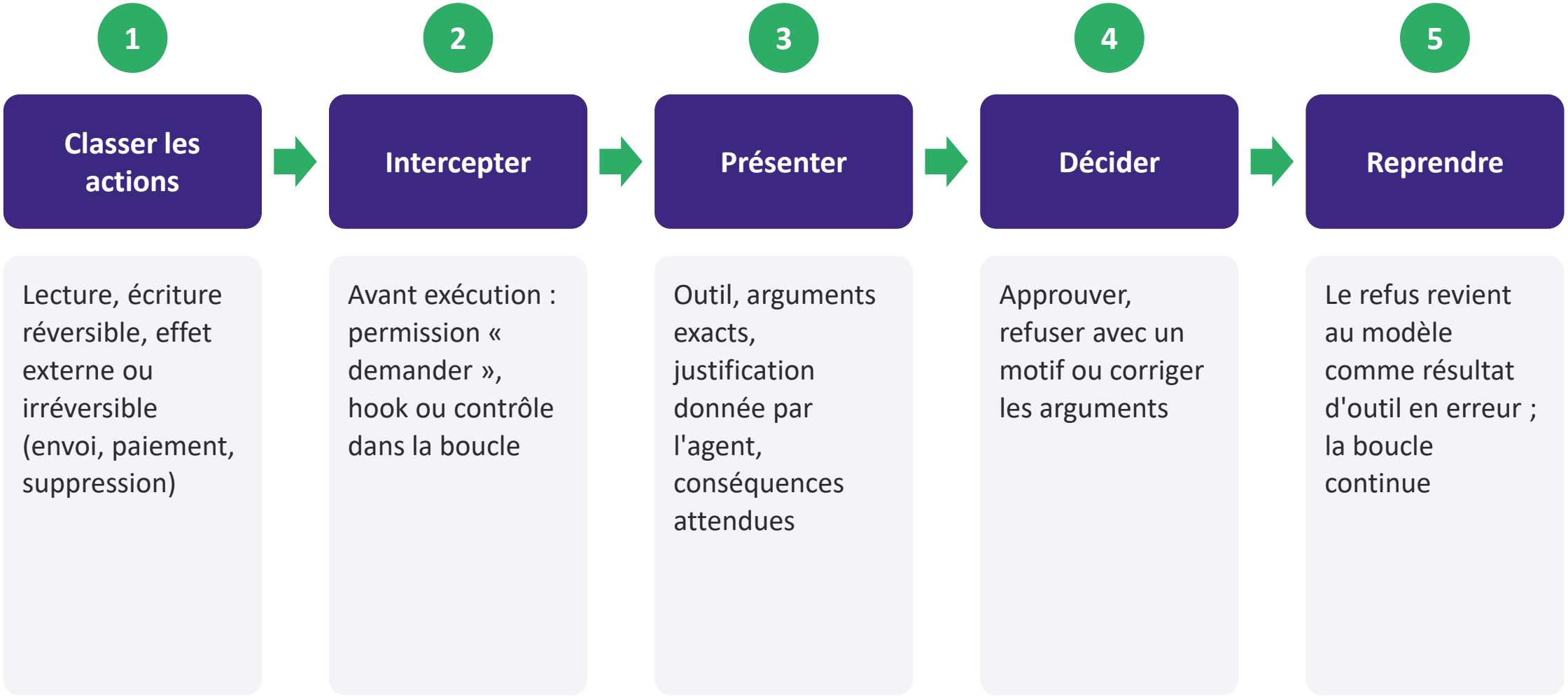
- Exposer uniquement les outils utiles à la tâche en cours
- Préférer des outils étroits (créer_brouillon) à des outils génériques (exécuter_sql, bash)
- Lecture seule par défaut ; écriture et suppression comme outils séparés
- Valider les arguments dans le code : types, plages, listes blanches
- Identifiants de service limités aux seules ressources nécessaires
- Mode de permission par outil : refuser, demander, autoriser

Isoler l'exécution

- Conteneur ou VM jetable pour le code et les commandes générés
- Système de fichiers limité au répertoire de travail
- Réseau sortant filtré par liste blanche de domaines
- Aucun secret dans l'environnement visible par l'agent
- Quotas : durée, mémoire, nombre d'appels par exécution
- Données de production atteintes via des outils, jamais en accès direct

bstorm TRAINING

Validation humaine : points d'arrêt



bstorm TRAINING

Injection de prompt directe et indirecte

Mécanisme

- Directe : l'utilisateur écrit lui-même des instructions qui contournent le prompt système
- Indirecte : instructions cachées dans une donnée lue par un outil (page web, courriel, PDF)
- Le modèle ne sépare pas de façon fiable données et instructions dans son contexte
- Risque maximal : données privées + contenu non fiable + canal de sortie vers l'extérieur
- Effets : fuite de données, action non voulue, mémoire ou fichiers empoisonnés

Parades

- Retirer au moins un des trois ingrédients du risque maximal
- Séparer : le modèle qui lit le contenu non fiable n'a pas d'outils d'action
- Valider les arguments : domaines, destinataires et chemins en liste blanche
- Délimiter les données externes (balises dédiées), sans s'y fier seul
- Approbation humaine sur toute action à effet externe
- Inclure des cas d'attaque dans le jeu d'évaluation

bstorm TRAINING

Évaluations : cas de test et notation

Notation	Principe	Atout	Limite
Par code	Assertions : format, valeur, outil, état final	Rapide, gratuite, reproductible	Rigide sur les réponses libres
Par modèle	Un modèle juge selon une grille explicite	Évalue texte libre, ton, pertinence	Coût, biais, à calibrer sur avis humains
Humaine	Expert du domaine sur un échantillon	Référence de qualité	Lente, coûteuse, peu fréquente
Trajectoire	Contrôle de la séquence d'appels d'outils	Détecte détours et appels inutiles	Plusieurs chemins valides possibles

bstorm TRAINING

Observabilité : contenu d'une trace

- ☑ Identifiant d'exécution, versions du code, du prompt et du modèle
- ☑ Chaque appel au modèle : messages envoyés, réponse, stop_reason, tokens, latence
- ☑ Chaque appel d'outil : nom, arguments, résultat, durée, erreur éventuelle
- ☑ Usage du cache : tokens écrits et tokens lus
- ☑ Décisions humaines : approbation, refus, motif
- ☑ Coût calculé par exécution et par tâche
- ☑ Format standard (OpenTelemetry) exporté vers un outil de visualisation
- ☑ Entrées et résultats d'outils conservés pour rejouer l'exécution

bstorm TRAINING

Erreurs composées, limites d'itérations

```python
MODELE = "claude-sonnet-5-5"
MAX_TOURS = 15
BUDGET_TOKENS = 300_000

def lancer_agent(client, messages, outils):
    consommes = 0
    for tour in range(MAX_TOURS):
        rep = client.messages.create(model=MODELE, max_tokens=8000,
                                     tools=outils, messages=messages)
        consommes += rep.usage.input_tokens + rep.usage.output_tokens
        messages.append({"role": "assistant", "content": rep.content})
        if rep.stop_reason != "tool_use":
            return rep                                  # fin normale
        if consommes > BUDGET_TOKENS:
            raise RuntimeError("Budget de tokens dépassé")  # arrêt contrôlé
        # executer_outils renvoie les tool_result, erreurs comprises
        messages.append({"role": "user", "content": executer_outils(rep)})
    raise RuntimeError(f"Aucune conclusion après {MAX_TOURS} tours")
```

Deux bornes indépendantes : nombre de tours et tokens consommés. Chaque arrêt est explicite et journalisé.

bstorm TRAINING

SECTION

1E

Du poste local au déploiement chez un client

Surfaces d'exécution, emballage, hébergement, déclencheurs, secrets, exploitation

Tableau comparatif des surfaces

Surface	Boucle gérée par	Outils	Hébergement	Accès et coût
API Messages brute	Code du développeur	Définis par le développeur + outils serveur	Libre : poste, serveur, cloud	Clé API, facturation au token
SDK client + tool runner	Assistant du SDK (bêta)	Fonctions Python décorées	Libre	Clé API, facturation au token
Agent SDK	Harnais Claude Code embarqué	Intégrés (fichiers, bash, web) + MCP	Processus ou conteneur du développeur	Clé API ou compte Claude (usage personnel)
Managed Agents	Infrastructure Anthropic	Bac à sable hébergé, MCP, outils client	Anthropic, ou bac à sable auto-hébergé	Clé API : tokens + temps de session (bêta)
Claude Code / Desktop	L'application	Code : intégrés, MCP, skills ; Desktop : MCP	Poste de l'utilisateur	Abonnement Claude ; clé API possible pour Code
Ollama (local)	Code client ou framework	Appel de fonctions selon le modèle	Machine locale ou serveur propre	Gratuit hors matériel et énergie

bstorm TRAINING

Pile technique d'un agent

Couche	Rôle	Exemples
Modèles	Raisonner, choisir l'action suivante	Claude, GPT, Gemini, Mistral, Llama
Outils et connecteurs	Agir sur les systèmes et y lire des données	Function calling, MCP, API REST, computer use, navigateur
Connaissance et mémoire	Retrouver documents et faits durables	RAG, embeddings, pgvector, Qdrant
Orchestration par le code	Boucle, état, outils, sous-agents	Agent SDK (Claude, OpenAI), LangGraph, CrewAI, AutoGen
Orchestration sans code	Enchaîner étapes et appels au modèle	n8n, Make : workflows visuels avec nœuds IA
Communication entre agents	Déléguer une tâche à un agent tiers	A2A : fiche d'agent, tâches, messages
Garde-fous et observabilité	Limiter, tracer, mesurer	Permissions, sandbox, traces, évals, Langfuse, LangSmith

bstorm TRAINING

API brute

Fourni

- Un point d'accès unique : POST /v1/messages
- Définition d'outils par schéma JSON, réponses tool_use structurées
- Outils serveur : recherche web, récupération de page, exécution de code
- Cache de prompt, sorties structurées, compaction côté serveur (bêta)
- Comptage des tokens, traitement par lots, API Files

À écrire

- La boucle : appel, exécution des outils, renvoi des résultats, arrêt
- Le stockage de l'historique entre les tours et entre les sessions
- La gestion des erreurs : outils en échec, reprises, délais
- Les permissions, validations et points d'arrêt humains
- Les traces, le suivi des coûts, les limites d'itérations
- L'hébergement, la montée en charge, la persistance

bstorm TRAINING

SDK client et tool runner

```python
import anthropic
from anthropic import beta_tool
client = anthropic.Anthropic()          # lit ANTHROPIC_API_KEY
STOCKS = {"A-1042": 12, "B-2001": 0}    # données de démonstration

@beta_tool
def stock_article(reference: str) -> str:
    """Renvoie le stock disponible pour une référence d'article.

    Args:
        reference: Référence interne, par exemple A-1042.
    """
    return str(STOCKS.get(reference, 0))
runner = client.beta.messages.tool_runner(
    model="claude-sonnet-5-5", max_tokens=4096, tools=[stock_article],
    messages=[{"role": "user", "content": "Reste-t-il des A-1042 ?"}])
for message in runner:                  # un message par tour, arrêt automatique
    print(message.content)
```

Le décorateur produit le schéma JSON à partir de la signature et de la docstring ; le runner exécute la boucle.

bstorm TRAINING

Agent SDK : abonnement ou clé API

```python
import asyncio
from claude_agent_sdk import ClaudeAgentOptions, ResultMessage, query
options = ClaudeAgentOptions(
    model="claude-sonnet-5-5",
    system_prompt="Analyste de journaux applicatifs. Réponses courtes.",
    tools=["Read", "Grep", "Glob"],          # outils intégrés disponibles
    allowed_tools=["Read", "Grep", "Glob"],  # exécutés sans confirmation
    cwd="./journaux",
    max_turns=10,
)

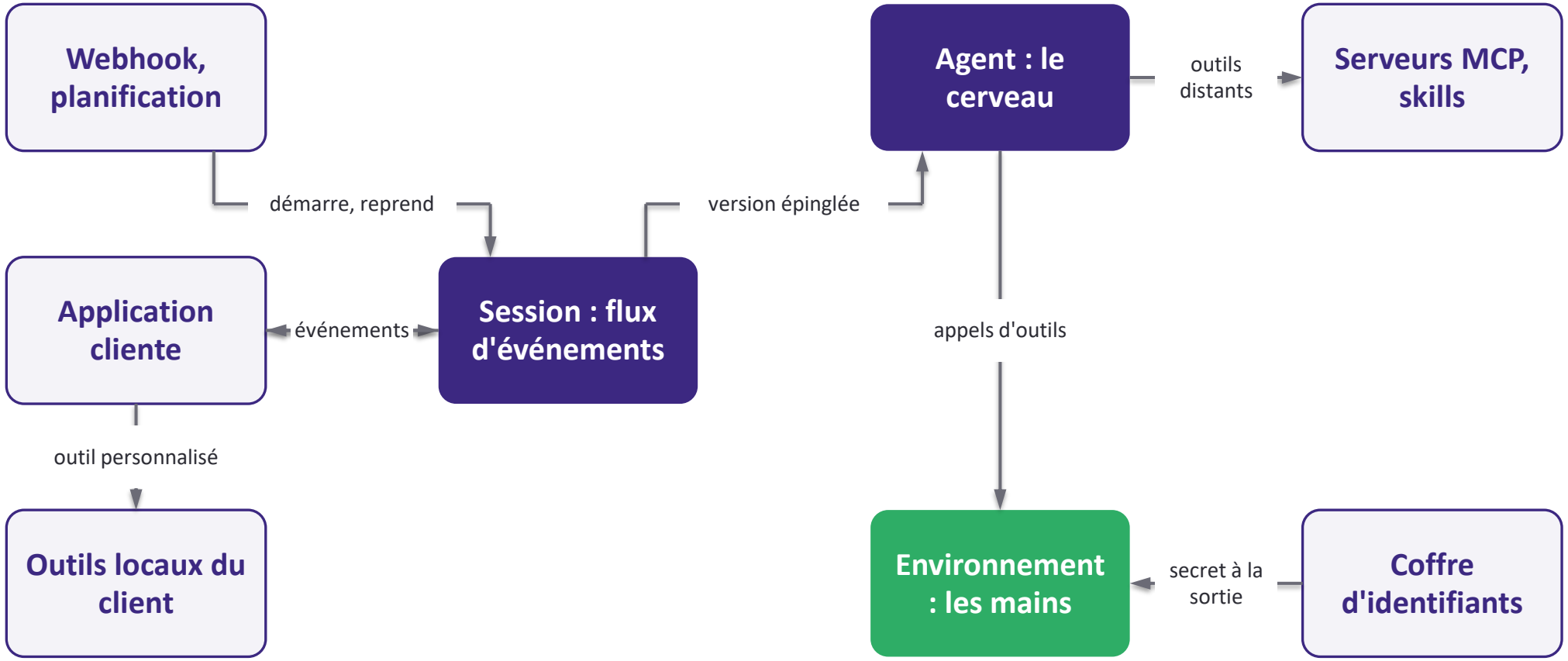
async def main():
    async for message in query(prompt="Quelles erreurs reviennent le plus ?",
                               options=options):
        if isinstance(message, ResultMessage):
            print(message.result, message.total_cost_usd, message.num_turns)

asyncio.run(main())
```

Sans model ni system_prompt, le SDK applique ses valeurs par défaut : mesure réelle d'environ 60 k tokens par requête.

bstorm TRAINING

Managed Agents : cerveau et mains



bstorm TRAINING

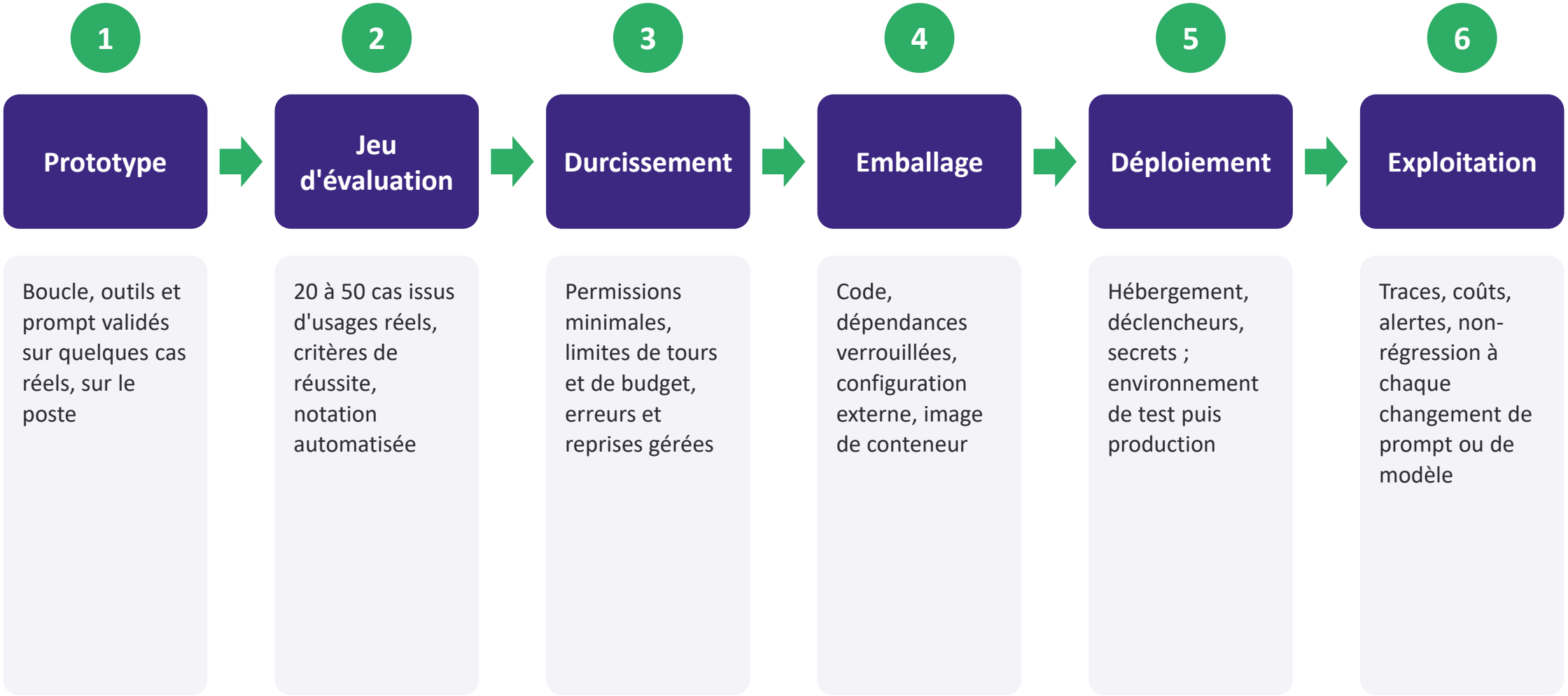
Claude Code, Claude Desktop, claude -p

Outil	Fonctionnement	Usage pour un projet d'agent
Claude Code interactif	Terminal ou IDE, boucle et outils intégrés	Prototyper, écrire et tester l'agent et ses outils
claude -p	Non interactif, sortie texte ou JSON	Scripts, intégration continue, tâches planifiées
Claude Code en CI	Action déclenchée par un événement du dépôt	Revue de PR, correctifs, tri de tickets
Claude Desktop	Application de conversation, hôte MCP	Tester un serveur MCP local, usage bureautique

Local : Ollama et modèles ouverts

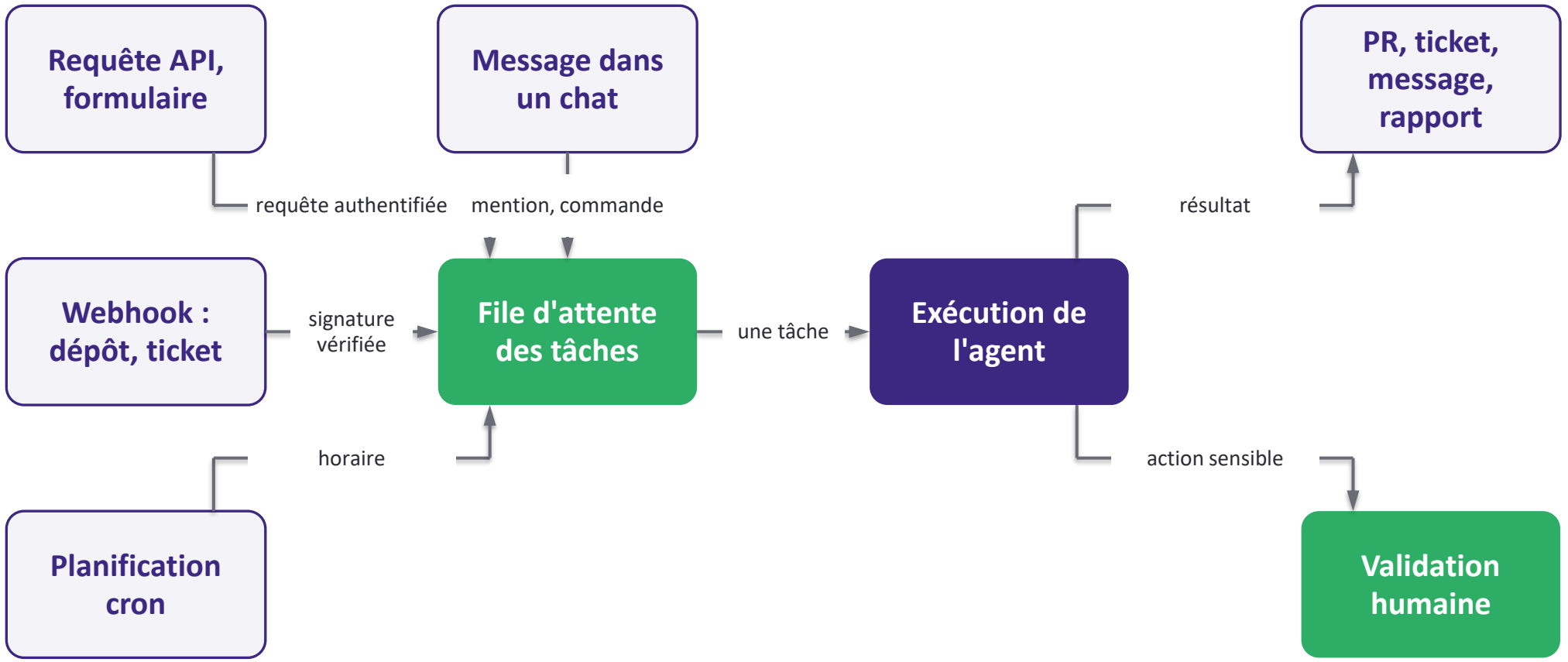
- Ollama : serveur local qui télécharge et exécute des modèles ouverts (gpt-oss, Qwen, Llama, Mistral…)
- API HTTP locale (port 11434), compatible avec le format de l'API OpenAI
- Appel de fonctions pris en charge par les modèles entraînés pour, fiabilité liée à leur taille
- Boucle à écrire, ou framework : Agents SDK d'OpenAI, Pydantic AI, smolagents
- Matériel : environ 16 Go de mémoire (GPU ou unifiée) pour un modèle de 20 milliards de paramètres
- Atouts : aucune donnée ne quitte la machine, coût marginal nul, fonctionnement hors ligne
- Limites : tâches longues moins bien tenues, sorties structurées moins fiables, lenteur sur CPU seul
- Usages : exercices, données qui ne peuvent pas quitter le site, tâches simples en volume

Cycle complet : du prototype à l'exploitation



bstorm TRAINING

Déclencheurs



bstorm TRAINING

Secrets et identités

- ☑ Aucun secret dans le code, le prompt ou l'image : variables d'environnement ou gestionnaire de secrets
- ☑ Une identité de service par agent et par environnement, distincte des comptes humains
- ☑ Droits minimaux sur chaque système appelé, jetons à portée et durée limitées
- ☑ Secrets hors du contexte du modèle : l'outil les injecte au moment de l'appel
- ☑ Clé API dédiée par client et par environnement, avec plafond de dépenses
- ☑ Rotation planifiée, révocation possible sans redéploiement
- ☑ Actions journalisées avec l'identité de l'agent et celle de l'utilisateur servi
- ☑ Fédération d'identité (OIDC) plutôt que clés longues quand la plateforme le permet

Exploitation : supervision, coûts, modèles, non-régression

- Supervision : taux de réussite, durée, tours par tâche, erreurs d'outils, alertes sur dérive
- Coûts : tokens par tâche et par client, part lue en cache, plafonds et alertes budgétaires
- Versions : prompt, outils et configuration versionnés avec le code, identifiant de modèle épinglé
- Changement de modèle : jeu d'évaluation rejoué, comparaison qualité, coût et latence avant bascule
- Dépréciations : calendrier de retrait des modèles et des API suivi, migration planifiée
- Non-régression : chaque modification de prompt, d'outil ou de modèle rejoue l'évaluation en CI
- Déploiement progressif : version candidate sur une part du trafic, retour arrière immédiat
- Amélioration : échec en production → nouveau cas d'évaluation → correction → mesure

bstorm TRAINING

SECTION

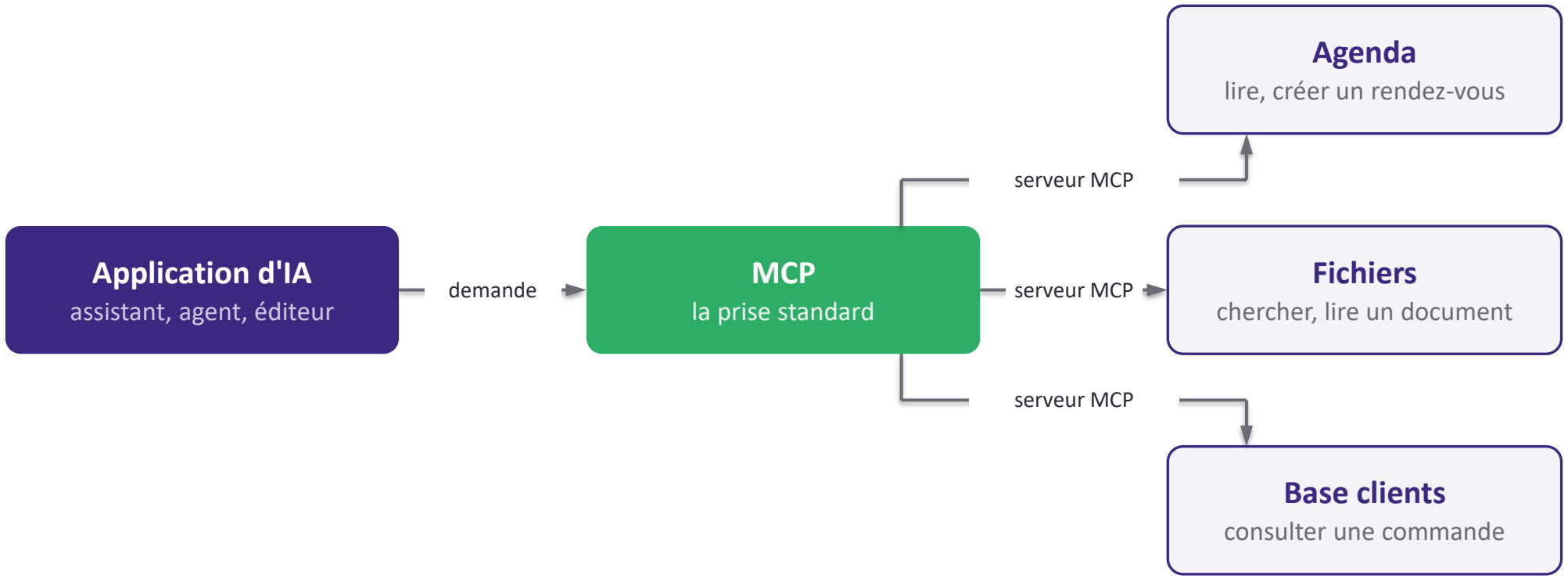
2

MCP

Model Context Protocol, spécification 2026-07-28

MCP en une image : une prise standard

Comme une prise USB-C : un même câble pour brancher n'importe quel appareil. MCP fait de même entre une IA et des outils.



bstorm TRAINING

Ce que MCP change concrètement

Exemple : demander à l'assistant « où en est la commande de M. Dupont ? » et obtenir la réponse issue de la vraie base clients.

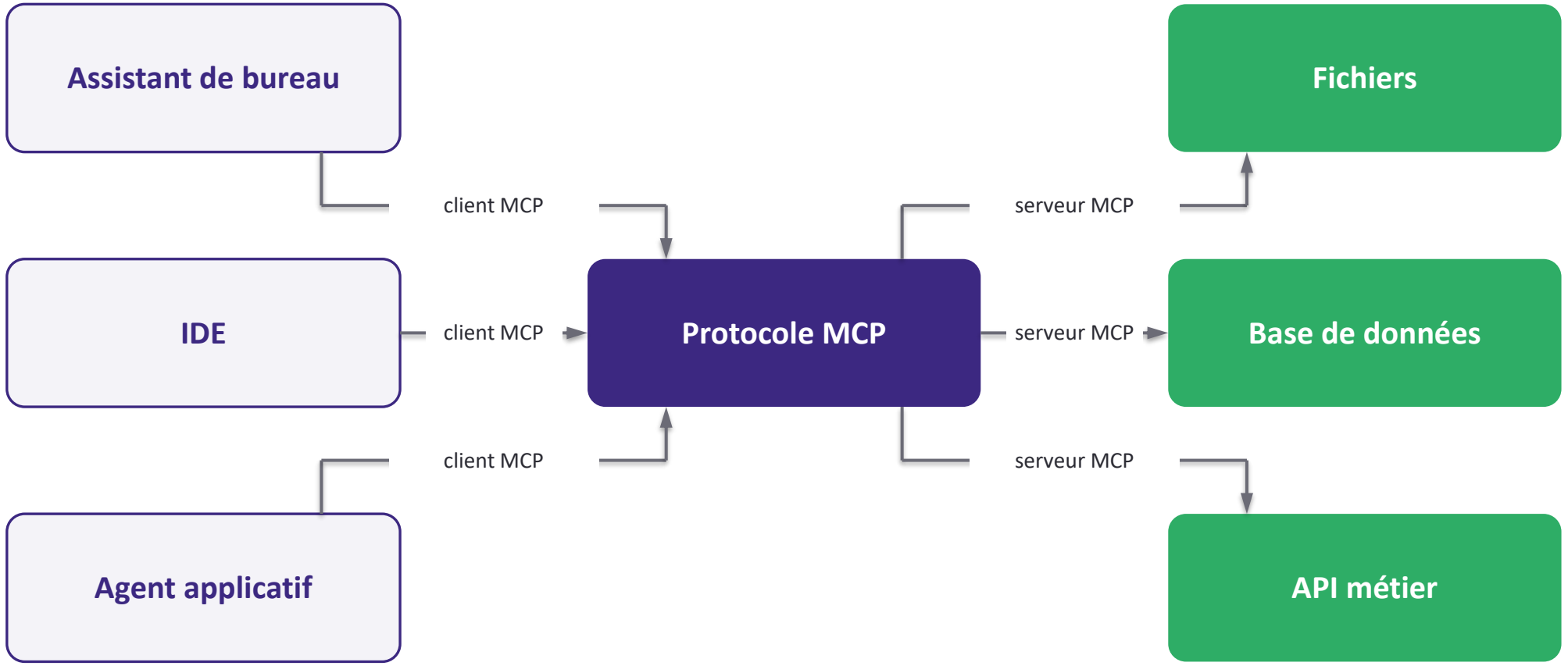
Sans MCP

- Copier-coller les informations dans la conversation
- Ou développer un connecteur propre à chaque application d'IA
- Un outil, cinq applications : cinq connecteurs à écrire et à maintenir
- Chaque éditeur invente son format

Avec MCP

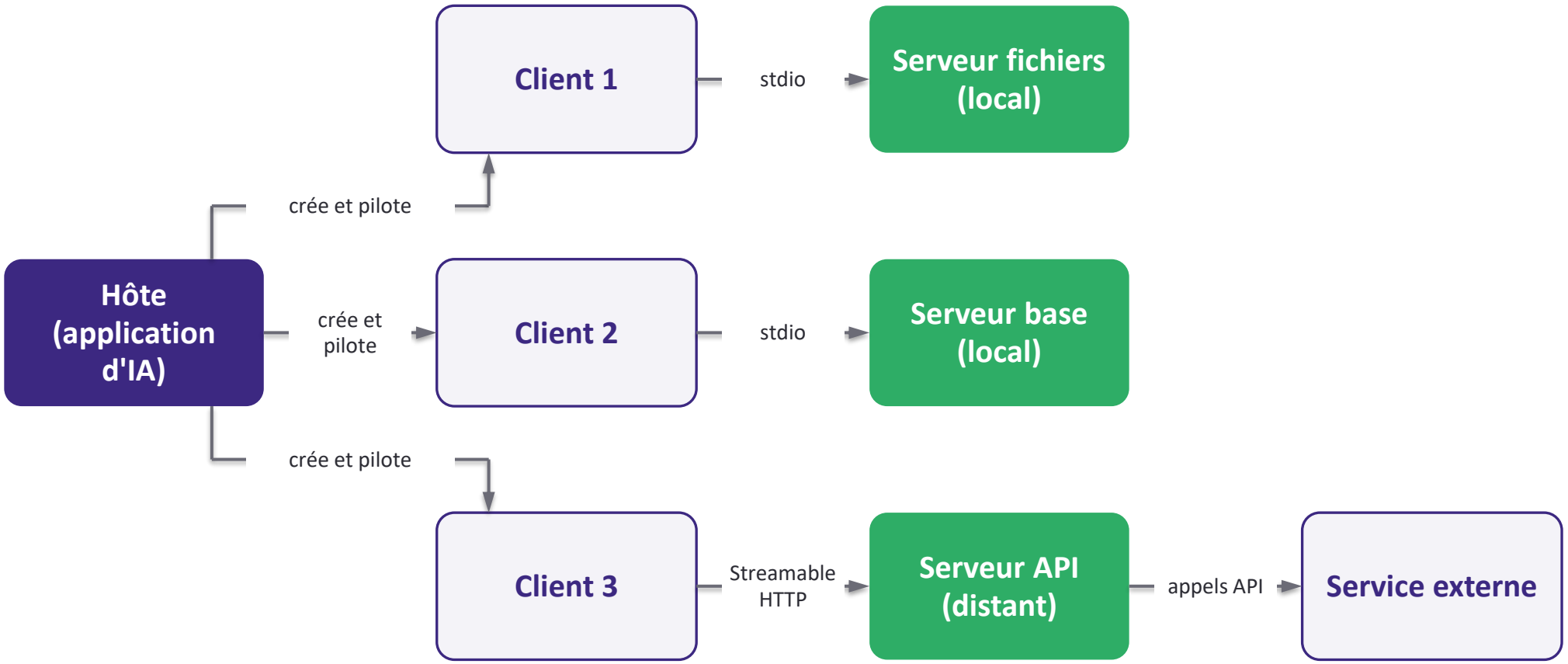
- L'assistant interroge lui-même la base clients via un serveur MCP
- Le serveur est écrit une fois et sert à toutes les applications compatibles
- L'utilisateur voit et autorise ce que l'assistant appelle
- Des serveurs existent déjà pour fichiers, GitHub, bases de données, navigateur

Problème N × M



bstorm TRAINING

Hôte, client, serveur



bstorm TRAINING

Outils, ressources, prompts

Primitive	Contrôle	Contenu	Méthodes
Outils (tools)	Le modèle	Fonctions qui agissent ou cherchent	tools/list, tools/call
Ressources	L'application	Données de contexte identifiées par URI	resources/list, resources/read
Prompts	L'utilisateur	Modèles de messages, commandes slash	prompts/list, prompts/get

bstorm TRAINING

Serveurs MCP courants

Exemples typiques : chaque serveur choisit ses primitives ; beaucoup n'exposent que des outils.

Serveur	Outils	Ressources	Prompts
Système de fichiers	Lire, écrire, lister, chercher dans les dossiers permis	Peu courant : outils seuls	Peu courant
Git, GitHub	Statut, diff, commit ; tickets, PR, revues	Fichiers du dépôt, d'une branche, d'une PR	Transformer un ticket en correctif
Base de données	Requête en lecture, description des tables	Schéma des tables	Analyser une table
Navigateur	Ouvrir une page, cliquer, saisir, lire la console	Peu courant	Peu courant
Agenda, messagerie	Chercher, créer un événement, rédiger un brouillon	Agenda du jour, fil de messages	Préparer une réunion
Outil métier interne	Opérations métier : client, commande, ticket	Fiches, référentiels, procédures	Commandes slash de l'équipe

bstorm TRAINING

JSON-RPC 2.0

```json
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "chercher_livres",
    "arguments": {"requete": "Herbert"},
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientCapabilities": {}
    }
  }
}
```

Réponse : même id et result avec resultType, ou error avec code et message. Notification : pas d'id, pas de réponse.

bstorm TRAINING

stdio

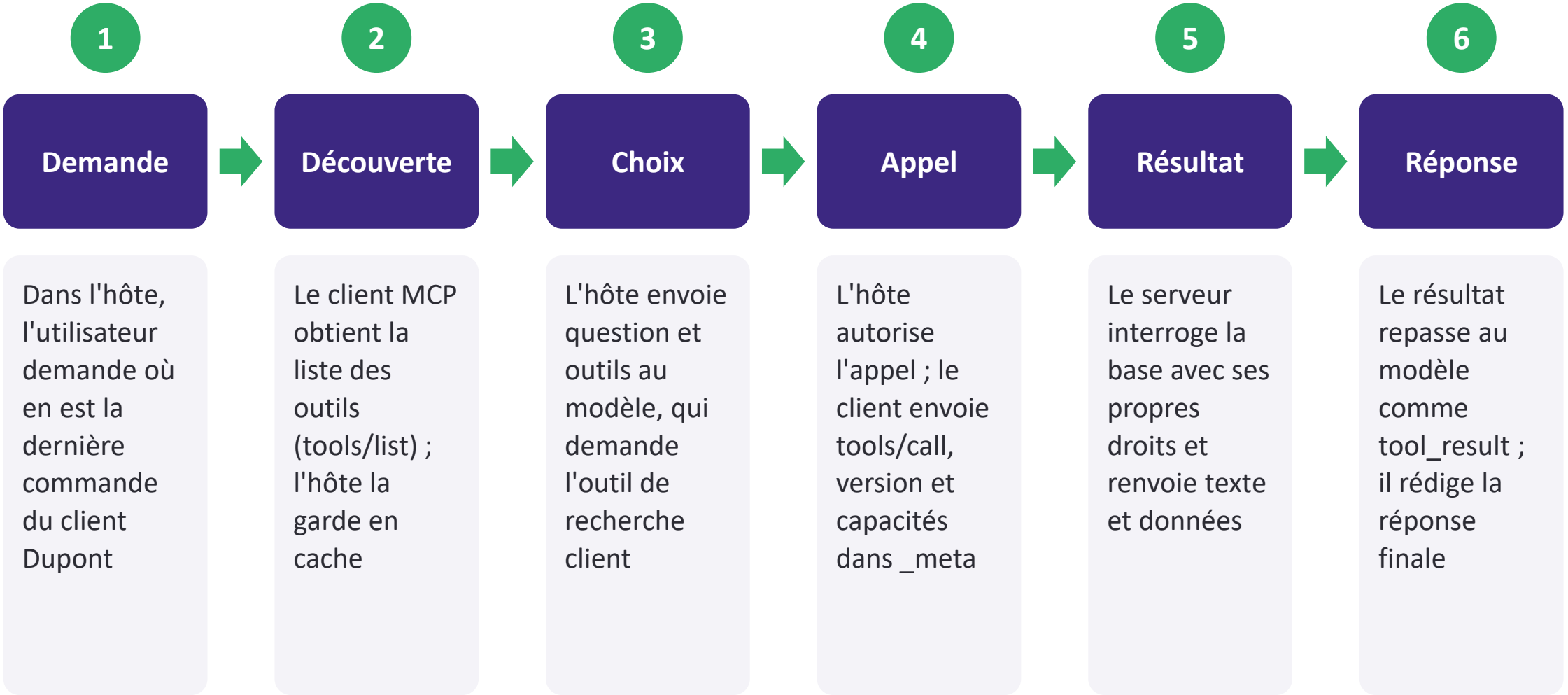
stdio (entrée et sortie standard)

Transport local : le client lance le serveur comme sous-processus. Messages JSON-RPC sur stdin et stdout, un par ligne, sans saut de ligne interne. stdout ne porte que des messages MCP ; les journaux vont sur stderr.

Exemple Arrêt : le client ferme stdin, attend la fin du processus, puis le termine de force s'il ne s'arrête pas.

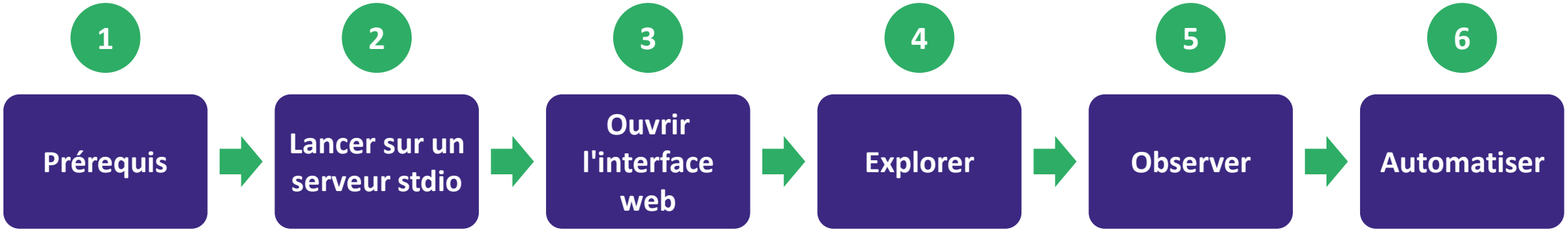
bstorm TRAINING

Scénario de bout en bout : un serveur « base clients »



bstorm TRAINING

MCP Inspector



Node.js 22.19.0 ou plus récent ; exécution directe par npx, sans installation.

Lancer npx avec le paquet inspector, puis la commande du serveur (uv run serveur.py).

L'URL affichée porte un jeton propre au lancement ; le navigateur s'ouvre.

Lister outils, ressources et prompts, appeler un outil avec des arguments, lire le résultat.

Le panneau de suivi affiche les échanges JSON-RPC bruts.

--cli avec --method tools/list pour la CI ; --tui pour le terminal ; --server-url en distant.

bstorm TRAINING

Sécurité MCP : empoisonnement d'outils, serveurs tiers

Menaces

- Empoisonnement : consignes cachées dans le nom ou la description d'un outil
- Shadowing : description qui oriente l'usage des outils d'un autre serveur
- Rug pull : définitions modifiées après l'approbation du serveur
- Serveur malveillant, trop gourmand en accès, ou dépendance compromise
- Serveur honnête détourné par une injection dans le contenu traité

Parades

- Lire les descriptions d'outils (Inspector) ; retirer tout serveur suspect
- Client qui enregistre les définitions et redemande en cas de changement
- Éditeur identifiable, code lisible, version épinglée
- Conteneur ou VM, montage limité au répertoire utile, sortie réseau filtrée
- Jeton propre à chaque serveur, portée minimale ; peu de serveurs actifs

bstorm TRAINING

MCP, outil natif ou skill

Critère	Outil natif	Serveur MCP	Skill
Nature	Fonction déclarée dans l'appel API	Programme qui expose des primitives	Instructions et fichiers chargés à la demande
Réutilisation	Une application	Tout hôte compatible MCP	Tout agent qui lit les skills
Coût en contexte	Schéma envoyé avec chaque requête	Définitions des outils du serveur	Description courte, corps lu si utile
Accès au système	Code de l'application	Code du serveur, avec ses droits	Outils déjà présents dans l'agent
Cas typique	Logique propre à un seul agent	Intégration partagée : SaaS, base, fichiers	Procédure, savoir-faire, conventions

bstorm TRAINING

Synthèse MCP

- MCP ramène N × M intégrations à N + M : une prise standard entre applications et outils
- Hôte, client, serveur ; outils (modèle), ressources (application), prompts (utilisateur)
- Messages JSON-RPC 2.0 : stdio en local, Streamable HTTP à distance
- Un serveur se teste sans modèle avec MCP Inspector
- Serveurs tiers et descriptions d'outils traités comme des entrées non fiables
- MCP, outil natif ou skill : choisir selon la réutilisation et le coût en contexte

bstorm TRAINING

Synthèse générale

- Agent : un modèle qui utilise des outils dans une boucle ; un workflow suffit souvent
- Le modèle est sans état : tout passe par le contexte, à construire, réduire et compacter
- Un outil = nom, description, schéma JSON ; sa qualité détermine celle de l'agent
- Patterns : chaînage, routage, parallélisation, orchestrateur-exécutants, évaluateur-optimiseur
- Fiabilité : garde-fous en couches, moindre privilège, points d'arrêt humains, bornes de boucle
- Démarrer en mode assisté ; évaluations, traces et coûts mesurés avant d'élargir l'autonomie
- MCP : protocole standard hôte, client, serveur ; outils, ressources, prompts
- Surfaces : de l'API brute au harnais hébergé, choix selon qui gère la boucle et l'hébergement
- Production : emballage, déclencheurs, secrets hors du contexte, non-régression à chaque changement

bstorm TRAINING